

APELLIDOS: _____ NOMBRE: _____

Bloque A

[0.75p] 1. Un programa consta de 10^9 instrucciones. El 60% son operaciones en punto flotante que tardan 10 ciclos en ejecutarse, mientras que las restantes tardan 5 ciclos. El programa se ejecuta en dos procesadores, S1 y S2, que implementan el mismo conjunto de instrucciones. S1 es un procesador multiciclo a 2 GHz y S2 un procesador segmentado a 500 MHz. En S2 el programa tarda 3 segundos en ejecutarse.

(a) Calcula el CPI de S1 y S2 para este programa.

$$CPI_{S1} = \frac{(0.6 \times 10 + 0.4 \times 5) \times 10^9 \text{ ciclos}}{1 \times 10^9 \text{ instr}} = 8 \text{ ciclos/instr}$$

El procesador segmentado puede solapar la ejecución de instrucciones, por lo que su CPI se reducirá. Al conocer el tiempo de ejecución se puede calcular el número de ciclos y por tanto el CPI.

$$CPI_{S2} = \frac{3 \text{ s} \times 0.5 \times 10^9 \text{ ciclos/s}}{10^9 \text{ instr}} = 1.5 \text{ ciclos/instr}$$

(b) Calcula el rendimiento de S1 según la métrica FLOPS.

$$T_{CPU} = \frac{(0.6 \times 10 + 0.4 \times 5) \times 10^9 \text{ ciclos}}{2 \times 10^9 \text{ ciclos/s}} = 4 \text{ s}$$

$$\frac{0.6 \times 10^9 \text{ flops}}{4 \text{ s}} = 0.15 \times 10^9 \text{ FLOPS} = 150 \text{ MFLOPS}$$

(c) Usando la Ley de Amdahl, determina en qué proporción debería mejorarse la unidad de punto flotante de S1 para que la ejecución de este programa tarde lo mismo en ambos procesadores.

$$\left. \begin{array}{l} T_{S1} = 4 \text{ s} \\ T_{S2} = 3 \text{ s} \end{array} \right\} A = \frac{4}{3}$$

$$f_m = \frac{0.6 \times 10}{0.6 \times 10 + 0.4 \times 5} = \frac{3}{4}$$

$$\frac{4}{3} = \frac{1}{1 - 3/4 + \frac{3/4}{A_m}}$$

$$A_m = \frac{3}{2} = 1.5$$

[2p] 2. Un computador utiliza memoria virtual paginada en dos niveles y cuenta con 32 KiB (2^{15} bytes) de memoria principal y 1 MiB (2^{20} bytes) de memoria virtual. La memoria se divide en 128 páginas físicas (2^7). Cada tabla de páginas involucrada en el proceso de traducción ocupa exactamente una página física, y cada entrada comienza con 1 bit de residencia, seguido de 1 bit de modificación, 23 bits adicionales de protección y control, y el número de página física.

- (a) Indica en qué campos y con qué valores (en hexadecimal) se dividiría la dirección virtual 0x5D392 y la dirección física 0x72B4.

Si los 2^{15} bytes de memoria se dividen en 2^7 páginas, el tamaño de página es 2^8 bytes. Cada entrada de una TP ocupa $1+1+23+7 = 32$ bits = 4 bytes. Cada tabla de páginas tiene $256/4 = 64$ entradas. Una DV (20 bits) se divide en 6 bits de PV1, 6 bits de PV2 y 8 bits de Δ_p . Una DF (15 bits) se divide en 7 bits de PF y 8 bits de Δ_p . Entonces las direcciones se dividen de la siguiente forma:

PV1	PV2	Δ_p	PF	Δ_p
01 0111	01 0011	1001 0010	0111 0010	1011 0100
0x17	0x13	0x92	0x72	0xB4
6 bits	6 bits	8 bits	7 bits	8 bits

- (b) Calcula cuánto ocuparían todas las tablas de nivel 2 de un proceso

Una DF (15 bits) se divide en 7 bits de PF y 8 bits de Δ_p . Cada entrada de una tabla contiene $1 + 1 + 23 + 7 = 32$ bits = 4 bytes. Cada tabla tiene $256/4 = 64$ entradas. Hay por tanto una tabla de nivel 1 y 64 tablas de nivel 2. Como cada una ocupa 256 bytes, ocupan en total $64 * 256 = 16$ KiB.

- (c) En la traducción de las siguientes direcciones virtuales (DV), se indica en la siguiente tabla el contenido de las tablas de páginas de nivel 1 (PT1) y 2 (PT2), en hexadecimal. Si es posible realizar la traducción, indica la dirección física correspondiente (DF). En caso contrario, justifica por qué no es posible hacerla.

DV	TP1	TP2	DF
905D7	8341 1C00	6529 AC32	no es posible
72A42	9B21 6B30	C2A8 04A5	0x2542
C3B12	D281 6A03	8A21 BC52	0x5212

En el caso indicado no es posible hacer la traducción porque el Bit de Residencia de la entrada en TP2 es 0. Para el resto, los 7 últimos bits de la entrada en TP2 indican la PF y concatenamos los 8 últimos bits de la DV.

- (d) Para la primera dirección del apartado anterior (0x905D7), determina la dirección de la entrada en la tabla de páginas de nivel 2.

La entrada en TP1 indica que la TP2 se encuentra en la página física 0. La DV indica que debemos acceder a la entrada 5 de esa tabla. Como cada entrada ocupa 4 bytes, la dirección es $0 + 5 * 4 = 0x0014$.

- (e) Supongamos que este sistema tuviese 256 KiB de memoria virtual y un proceso utiliza todo el espacio virtual. Las direcciones virtuales se dividirían en 10 bits de página virtual y 8 bits de desplazamiento. Determina la partición más eficiente de esos 10 bits de PV en dos niveles de paginación.

Tendremos una TP de nivel 1 y tantas TP de nivel 2 como entradas tenga la TP de nivel 1. Teniendo en cuenta que las tablas ocuparán siempre una página (2^8 bytes), para minimizar el espacio total lo ideal es que haya el menor número de TP de nivel 2 y estén completas (2^6 entradas) para que no exista fragmentación interna. Por tanto, la mejor opción es 4 bits de PV1 y 6 de PV2.

APELLIDOS: _____ NOMBRE: _____

Bloque A

- [0.5p] 3. Tenemos discos duros de capacidad 2 TiB y queremos montar un RAID con una capacidad neta de 10 TiB. Estamos pensando en las configuraciones de RAID 5 y RAID 6. Razona cuál sería la capacidad total que tendría cada una de estas dos configuraciones. Como sistemas RAID, indica dos características comunes y una diferencia entre un RAID 5 y un RAID 6.

Un **RAID 5** utiliza el espacio de un disco para almacenar una función de redundancia, por tanto, tendría una capacidad total de **12 TiB**, mientras que un **RAID 6** implementa dos funciones de redundancia, así que necesita el espacio de dos discos, con lo cual su capacidad total sería de **14 TiB**.

Características comunes entre un RAID 5 y un RAID 6:

- División de datos en tiras de varios bytes.
- Reparto de datos redundantes cíclicamente entre todos los discos.
- Posibilidad de varias operaciones de E/S concurrentes.

Diferencias entre un RAID 5 y un RAID 6:

- Un RAID 5 implementa una sola función de redundancia, mientras que un RAID 6 implementa dos.
- Un RAID 5 puede recuperarse de un fallo de disco, mientras que un RAID 6 puede recuperarse de dos.

[0.75p] 4. Tenemos un sistema con memoria virtual de páginas de 1 KiB que conecta la memoria principal con el almacenamiento secundario mediante un bus síncrono de 128 bits a 1 GHz.

Las transferencias de las páginas están gestionadas por una DMA a través de este bus en bloques del tamaño de la anchura del bus, necesitando 300 ns para la lectura del primer bloque y 10 ns para el resto. La DMA realiza la lectura de una página en una única transacción.

Tanto el envío de 128 bits a través del bus como el de la dirección al controlador del almacenamiento secundario requieren un ciclo. Las transferencias de datos leídos más recientemente pueden solaparse con la lectura de los siguientes.

Para finalizar la transacción la CPU atiende la interrupción enviada por la DMA ejecutando una rutina de 17 instrucciones. La CPU es multiciclo y cada instrucción necesita 4 ciclos para ejecutarse, siendo la frecuencia de CPU también de 1 GHz.

Calcula la latencia y el ancho de banda para la lectura de una página.

El ciclo de reloj del bus y de la CPU es:

$$T_{\text{ciclo}} = \frac{1}{f} = \frac{1}{1 \text{ GHz}} = 10^{-9} \text{ s} = 1 \text{ ns}$$

El sistema soporta transferencias de 16 B (128 bits). Cada página son 1 KiB, que corresponden a $1 \text{ KiB} / (16 \text{ B/transf.}) = 2^6$ transferencias = 64 transferencias en una transacción.

Para leer 1 página necesitamos:

- 1 ciclo para enviar la dirección: 1 ns.
- Un tiempo de acceso a memoria de 300 ns para leer los primeros 16 B.
- 10 ns para acceder a cada uno de los 63 siguientes bloques de 16 B que se solapan con el envío del anterior que se ha leído (1 ns).
- 1 ciclo de envío para la última palabra (1 ns).
- Al finalizar una transacción se ejecuta la rutina que atiende la interrupción de finalización, que consta de 17 instrucciones, cada una de 4 ciclos (4 ns) de duración.

Por tanto, para la lectura de una página necesitamos:

$$1 + 300 + 63 \times 10 + 1 + 17 \times 4 = 1000 \text{ ns/transacción.}$$

Así, la **latencia** para la lectura de una página es de 1000 ns, y el ancho de banda:

$$AB = \frac{1 \text{ KiB}}{1000 \times 10^{-9} \text{ s}} \simeq 10^9 \text{ B/s} = 1 \text{ GB/s}$$

Bloque B

- [2p] 5. El siguiente código se ejecuta en el procesador segmentado de 5 etapas estudiado en clase: IF, ID, EX, MEM y WB. Los saltos se deciden en la etapa ID empleando predicción de salto fijo no efectivo. Hay una unidad de detección de riesgos en la etapa ID, y una unidad de anticipación en la etapa EX. La unidad de suma en punto flotante está segmentada, y requiere 3 ciclos de reloj para efectuar una operación de suma. Cada una de las etapas MEM y WB sólo puede gestionar una instrucción en cada ciclo.

- (a) [0.5p] Muestra el diagrama multiciclo para **la primera iteración** de este código, marcando de forma explícita **las anticipaciones y los bloqueos del pipeline**.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
1 addi \$t4, \$0, 10	IF	ID	EX	MEM	WB									
Loop:														
2 lw \$a1, 0(\$a0)		IF	ID	EX	MEM	WB								
3 add \$t0, \$a1, \$t4			IF	ID	ID	EX	MEM	WB						
4 sw \$t0, 0(\$a0)				IF	ID	ID	EX	MEM	WB					
5 addi \$t4, \$t4, -1						IF	ID	EX	MEM	WB				
6 addi \$a0, \$a0, 4							IF	ID	EX	MEM	WB			
7 bne \$t4, \$0, Loop								IF	ID	ID	EX	MEM	WB	
8 sw \$t0, 0(\$a0)									IF	IF	nop			

- (b) [0.25p] Indica dos dependencias verdaderas (de datos) y una antidependencia que aparezcan en el cuerpo del bucle.

Verdaderas: \$a1 entre la 3 y la 2 y \$t0 entre la 4 y la 3. Antidependencia: \$t4 entre la 5 y la 3

- (c) [0.25p] ¿Cuáles de estas dependencias generan un riesgo? ¿De qué tipo?

La dependencia entre la 3 y la 2 genera un riesgo RAW al tener que leer el dato de memoria en la etapa MEM de la instrucción 2 para poder anticiparlo a la etapa EX de la instrucción 3.

- (d) [0.25p] Discute la veracidad de la siguiente afirmación: “Si el procesador implementase salto retardado, se puede eliminar la instrucción 4 y, tras la ejecución, el array que recorre \$a0 tendrá los mismos valores que usando el código original en el procesador sin salto retardado, sólo que desplazados una posición.”.

Es falso. El resultado del código sería totalmente diferente, porque la instrucción 8 del hueco de retardo sobrescribirá en cada iteración el valor que leerá la instrucción 2 en la siguiente iteración.

- (e) [0.25p] Calcula la tasa de acierto del predictor de salto según los siguientes modos de procesamiento de salto que podría implementar el procesador: salto fijo no efectivo, predictor de salto de 1 bit y predictor de salto de 2 bits. En los modos de predicción dinámica, la predicción inicial es de salto no efectivo.

El salto se ejecuta 10 veces. Con salto fijo no efectivo sólo se acierta en la última ejecución: $TA = 1/10$. Con un predictor de 1 bit se fallará en la primera y la última ejecución del salto: $TA = 8/10$. Con un predictor de 2 bits se fallará en las 2 primeras y en la última ejecución: $TA = 7/10$.

- (f) [0.5p] Razona cuáles de las siguientes señales de control estarían activas (valor 1) en cualquiera de las etapas del procesado durante el ciclo de ejecución 8: MemRead, MemWrite, RegWrite, Mem2Reg, Branch, IF_flush.

WB: RegWrite (add), MEM: MemWrite (sw). El resto no se encuentran activas en el ciclo 8.

- [2p] 6. Un computador tiene una memoria principal de **256 KiB** y una memoria caché de **correspondencia directa** de **1024 B** con líneas de **4 bytes**. Las políticas de escritura son **post-escritura (write-back)** y **ubicar en escritura (write-allocate)**. Comenzando con la caché vacía, ejecutamos el siguiente código. La matriz A está almacenada por filas comenzando en la dirección 0x06000, y cada elemento de tipo “int” ocupa 4 bytes. Las variables escalares se almacenan en registros de la CPU y no generan accesos a memoria.

```
int i, j, A[7][1024];
for (i = 1; i <= 2; i++)
    for (j = 1; j <= 2; j++)
        A[j][i] = A[0][1024 - j]
```

- (a) [0.3p] Calcula la dirección de memoria ocupada por A[j][i] en función de “j” y “i”.

$$A[i][j] = 0x06000 + 0x1000 * j + 0x4 * i$$

- (b) [0.3p] Indica la partición en campos de una dirección física desde el punto de vista de la caché.

Etiqueta (8 bits), Índice (8 bits), Desplazamiento (2 bits)

- (c) [0.6p] Rellena todos los campos de la siguiente tabla de accesos a memoria: dirección, etiqueta, e índice/conjunto (en formato **hexadecimal**); indica si el acceso resulta en un acierto o un fallo caché, y en este último caso en qué tipo de fallo; e indica si el acceso provoca una escritura a memoria principal (MP) o no.

Dirección	Etiqu.	Idx	A/F	Tipo de Fallo	Escritura MP
06FFC	1B	FF	F	Forzoso	
07004	1C	1	F	Forzoso	
06FF8	1B	FE	F	Forzoso	
08004	20	1	F	Forzoso	SI
06FFC	1B	FF	A	Acierto	
07008	1C	2	F	Forzoso	
06FF8	1B	FE	A	Acierto	
08008	20	2	F	Forzoso	SI

carga 06FFC en set FF

carga+escribe 07004 en set 1

carga 06FF8 en set FE

reemplaza 07004 por 08004 (mod) en set 1

carga+escribe 07008 en set 2

reemplaza 07008 por 08008 (mod) en set 2

- (d) [0.3p] Si el tiempo medio de acceso a memoria fue de 43 ciclos, y el tiempo de acierto caché es de 1 ciclo, calcula el tiempo de acceso a memoria principal.

$$43 = 1 + \frac{6}{8} \times TMP \text{ ciclos} \rightarrow TMP = (43 - 1) \times \frac{8}{6} = 7 \times 8 = 56 \text{ ciclos}$$

- (e) [0.6p] Discute la veracidad de las siguientes afirmaciones:

- En esta caché se podría solapar la comprobación de la etiqueta con la transmisión de los datos
Sí, porque los datos sólo pueden proceder de la única línea del conjunto.
- Sólo los fallos de conflicto pueden provocar escrituras a memoria principal en esta caché.
No, cualquier fallo de cualquier tipo que expulse una línea modificada provocará una escritura a memoria.